UNIVERSIDAD DE COSTA RICA
Sede del Atlántico

RP Recinto de Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Quiz 9: Auditoría Forense de Vulnerabilidades Web (OWASP Top 10) en InsecureCare

(Solucionario Oficial y Justificación Pedagógica para el Docente)

Puntuación Total: 25 Puntos (5 Puntos por pregunta)
Tiempo Límite Estimado: 35 - 45 minutos
Modalidad: Evaluación Individual Práctica y Formativa / Sumativa
Escenario de Trabajo: Aplicación Web `InsecureCare` ubicada en `web_apps/unsecure_web_app/`

Cuestionario de Evaluación

1. Inventario y Cuantificación de Vulnerabilidades OWASP (5 Puntos)

Tras auditar exhaustivamente la arquitectura Full-Stack de la aplicación `InsecureCare` (revisando endpoints REST, persistencia JPA, configuración de Spring Boot y componentes Standalone en Angular), ¿cuál es la cantidad exacta de vulnerabilidades críticas del estándar **OWASP Top 10** deliberadamente implementadas en el

sistema y cuál es su clasificación correspondiente?

- [ ] A) Existen exactamente **3** vulnerabilidades: Inyección SQL (A03), Desbordamiento de Búfer (A04) y Fallo de Cifrado en Disco (A02).
- [x] B) Existen exactamente **5** vulnerabilidades: Broken Access Control / IDOR (A01), Fallos Criptográficos / Contraseñas en Texto Plano (A02), Inyección SQL (A03), Cross-Site Scripting Almacenado (A03) y Configuración de Seguridad Incorrecta / Fuga de Trazas (A05).
- [ ] C) Existen exactamente **2** vulnerabilidades: Cross-Site Scripting (XSS) y Denegación de Servicio Distribuida (DDoS) por falta de balanceador Nginx.
- [ ] D) Existen exactamente **7** vulnerabilidades: Ataque de Hombre en el Medio (MitM), Falsificación de Solicitudes del Servidor (SSRF), XML External Entity (XXE), Clickjacking, Broken Object Level Authorization, Memory Leak y Buffer Overflow.

---

2. Inyección SQL (SQLi) y Exfiltración de Registros Confidenciales (5 Puntos)

En la barra de búsqueda de diagnósticos clínicos del componente `ExpedienteListComponent`, un auditor de seguridad prueba evadir el filtro ingresando cadenas de control SQL en la URL:
`GET /api/expedientes/buscar?termino=' OR '1'='1`

¿Por qué este vector de ataque tiene éxito en el servidor y cuál es la solución técnica estándar en Spring Data JPA para neutralizarlo definitivamente?

- [ ] A) El ataque tiene éxito porque H2 Database no admite claves foráneas; se soluciona migrando el backend a Oracle Database sin tocar el código Java.
- [x] B) El ataque tiene éxito porque el servicio `ExpedienteService` concatena dinámicamente el parámetro del usuario en una consulta SQL nativa (`createNativeQuery`); se soluciona utilizando consultas parametrizadas con marcadores de posición (`:termino`) o métodos derivados de `JpaRepository`.
- [ ] C) El ataque tiene éxito porque el navegador envía los parámetros en texto claro mediante HTTP; se soluciona instalando un certificado SSL autofirmado en el puerto 4200 de Angular.
- [ ] D) El ataque tiene éxito debido a que la base de datos corre en memoria RAM; se neutraliza configurando `spring.datasource.username=root`.

---

3. Broken Access Control / Insecure Direct Object Reference (IDOR / BOLA) (5 Puntos)

El estudiante inicia sesión con las credenciales del paciente `carlos_mora` (Usuario ID: 3, asignado al Expediente #1). Al navegar a la vista de detalle, la URL del navegador muestra `/expediente/1`. Si el estudiante reemplaza manualmente la ruta por `/expediente/2`, ¿qué ocurre técnica y conceptualmente en la aplicación?

- [ ] A) El sistema lanza inmediatamente un error HTTP 403 Forbidden y bloquea la IP del cliente durante 24 horas por violación de políticas de seguridad.
- [x] B) Se produce una vulnerabilidad de Broken Access Control (IDOR / BOLA) donde el paciente accede al expediente psiquiátrico confidencial de `maria_rojas`, debido a que `ExpedienteService.obtenerPorId()` busca directamente por clave primaria sin verificar que el `usuarioId` del recurso coincida con el usuario autenticado en la sesión.
- [ ] C) El servidor Angular redirige al usuario a la página de inicio porque el componente Standalone detecta la discrepancia en el árbol de enrutamiento mediante un `CanActivateFn`.
- [ ] D) La base de datos H2 se corrompe automáticamente al recibir una petición de lectura fuera del ámbito transaccional `@Transactional`.

---

4. Cross-Site Scripting (XSS Almacenado) y Bypass en Angular (5 Puntos)

En el registro de un nuevo expediente médico, un atacante ingresa el siguiente contenido en el campo de observaciones clínicas:

```
<img src="invalido"
     onerror="alert('XSS Detectado')">
```

Al visualizar dicho registro en `ExpedienteDetailComponent`, la ventana modal de alerta de JavaScript se ejecuta exitosamente en el navegador. ¿Cuál fue el fallo cometido en el código del Front-End que provocó la ejecución del payload?

- [x] A) El desarrollador utilizó la directiva `[innerHTML]` invocando explícitamente `sanitizer.bypassSecurityTrustHtml()`, desactivando voluntariamente el motor de sanitización contextual nativo de Angular.
- [ ] B) Angular carece de defensas nativas contra XSS y requiere obligatoriamente instalar la librería externa jQuery para filtrar etiquetas peligrosas.
- [ ] C) El navegador web tiene deshabilitado el motor V8, provocando que las etiquetas `<img>` ejecuten scripts en el hilo de renderizado visual.
- [ ] D) El fallo ocurrió porque el archivo `index.html` no tiene configurado el atributo `lang="es"`.

---

5. Fallos Criptográficos y Fuga de Información Técnica (5 Puntos)

Durante la inspección de la base de datos mediante la consola `/h2-console` y al forzar un error de sintaxis SQL en la API, el equipo auditor observa dos violaciones críticas a la confidencialidad. ¿Cuáles son estas dos debilidades y qué directivas de configuración corrigen la fuga de información?

- [ ] A) El puerto 8080 está bloqueado por el firewall de Windows y la memoria JVM es insuficiente para procesar el garbage collector.

- [x] B) Las contraseñas de los usuarios se almacenan en texto plano en la tabla `USUARIOS`, y el servidor tiene configurado `server.error.include-stacktrace=always`, lo cual expone la arquitectura interna y consultas SQL; se remedia implementando `BCryptPasswordEncoder` y configurando `server.error.include-stacktrace=never`.
- [ ] C) El servidor utiliza algoritmos cuánticos no estandarizados; se soluciona deshabilitando el protocolo HTTP 1.1 en `pom.xml`.
- [ ] D) La consola de H2 requiere autenticación biométrica y el backend rechaza peticiones JSON de Angular.

Plantilla de Reporte de Hallazgos (Pentest Findings Sheet)

Espacio para ser completado por el estudiante durante la sesión práctica de auditoría:

No.	Nombre del Hallazgo / Vulnerabilidad	Categoría OWASP Top 10	Endpoint o Archivo Vulnerable	Evidencia / Payload de Explotación	Propuesta de Remediación
1					
2					
3					
4					
5					

Solucionario Analítico y Justificación Pedagógica

Pregunta 1: Inventario y Cuantificación de Vulnerabilidades OWASP

- **Opción Correcta: B**
- **Justificación de la Opción Correcta:**
  La arquitectura de `InsecureCare` fue construida con exactamente **5 fallos intencionados**:
- **A01:2021 - Broken Access Control / IDOR:** Endpoint `/api/expedientes/{id}` sin verificación de pertenencia del recurso (*Resource Ownership*).

- **A02:2021 - Cryptographic Failures:** Contraseñas en texto claro en `Usuario.java` y `data.sql`, junto a un secreto JWT débil (`ClaveSecretaSuperDebil...`).
- **A03:2021 - Injection (SQLi):** Concatenación de cadenas en `ExpedienteService.buscarPorTermino()` ejecutando SQL dinámico crudo.
- **A03:2021 - Injection (Stored XSS):** Inserción de código JavaScript arbitrario en `observacionesHtml` renderizado con bypass de sanitización.
- **A05:2021 - Security Misconfiguration:** Exposición de stack traces en `application.properties` (`server.error.include-stacktrace=always`) y consola H2 pública.
- **Análisis de Opciones Incorrectas:**
- **A es incorrecta:** Desbordamiento de búfer y cifrado en disco no aplican al diseño de esta aplicación web.
- **C es incorrecta:** Subestima severamente las vulnerabilidades del backend e inventa un ataque DDoS.
- **D es incorrecta:** Enumera vulnerabilidades inexistentes en esta arquitectura (no hay procesadores XML para XXE ni SSRF).

Pregunta 2: Inyección SQL (SQLi) y Exfiltración de Registros Confidenciales

- **Opción Correcta: B**
- **Justificación de la Opción Correcta:**
  En `ExpedienteService.java`, el método `buscarPorTermino` implementa: `java String sql = "SELECT * FROM EXPEDIENTES_MEDICOS " + "WHERE diagnostico LIKE '%" + termino + "%' ..."; entityManager.createNativeQuery(sql, ...);` Al ingresar `' OR '1'='1`, la consulta resultante se convierte en: `WHERE diagnostico LIKE '%' OR '1'='1%'`, convirtiendo la cláusula en una tautología verdadera para todas las tuplas de la tabla y exfiltrando todos los expedientes confidenciales. La remediación consiste en utilizar parámetros preparados: `java @Query("SELECT e FROM ExpedienteMedico e " + "WHERE e.diagnostico LIKE :filtro") List<ExpedienteMedico> buscar( @Param("filtro") String filtro);`
- **Análisis de Opciones Incorrectas:**
- **A es incorrecta:** El fallo radica en la lógica del código Java, no en el motor relacional subyacente.
- **C es incorrecta:** HTTPS protege el canal contra escuchas no autorizadas, pero el payload llega íntegro a la base de datos.
- **D es incorrecta:** Modificar el nombre de usuario de la BD no previene la concatenación de cadenas maliciosas.

Pregunta 3: Broken Access Control / IDOR (BOLA) en Registros Clínicos

- **Opción Correcta: B**
- **Justificación de la Opción Correcta:**
  La vulnerabilidad de Referencia Directa Insegura a Objetos (IDOR / BOLA) ocurre cuando el sistema

expone un identificador interno de base de datos (`id = 2`) en una interfaz o API sin validar que la identidad autenticada tenga privilegios legítimos sobre ese objeto en particular. Carlos Mora (`usuarioId = 3`) puede ver el expediente de María Rojas (`usuarioId = 4`) porque el backend recupera el registro únicamente con `expedienteRepository.findById(id)`. La corrección requiere validar la pertenencia: `java if (!expediente.getUsuario().getId() .equals(usuarioAutenticado.getId()) && !usuarioAutenticado.esMedico()) { throw new AccessDeniedException( "Acceso denegado al expediente ajeno."); }`

- **Análisis de Opciones Incorrectas:**
- **A es incorrecta:** InsecureCare carece intencionalmente de ese control defensivo.
- **C es incorrecta:** La aplicación carece de Guards `CanActivateFn` que bloqueen la navegación por parámetros de ruta.
- **D es incorrecta:** H2 ejecuta las consultas SQL de lectura concurrentes sin problemas de corrupción física.

Pregunta 4: Cross-Site Scripting (XSS Almacenado) y Bypass en Angular

- **Opción Correcta: A**
- **Justificación de la Opción Correcta:**
  Angular incorpora un sistema de seguridad de diseño defensivo que trata todo valor como no confiable por defecto, codificando caracteres peligrosos (`<`, `>`, `"`, `'`) en entidades HTML. Sin embargo, en `ExpedienteDetailComponent.ts`, el desarrollador invocó deliberadamente: `typescript this.observacionesSeguras = this.sanitizer.bypassSecurityTrustHtml( data.observacionesHtml);` Esto le indica al motor de Angular que suspenda el análisis de seguridad para ese fragmento de texto, permitiendo que navegadores ejecuten el manejador de eventos malicioso `onerror` de la imagen. La remediación consiste en omitir el bypass y emplear la directiva estándar o utilizar una librería de sanitización estricta como DOMPurify.
- **Análisis de Opciones Incorrectas:**
- **B es incorrecta:** Angular cuenta con uno de los sistemas de sanitización contextual más avanzados del mercado (`DomSanitizer`).
- **C es incorrecta:** El motor V8 ejecuta JavaScript legítimamente al dispararse el evento del DOM.
- **D es incorrecta:** El atributo `lang` en HTML solo tiene implicaciones de accesibilidad y SEO, no de seguridad contra scripts.

Pregunta 5: Fallos Criptográficos y Fuga de Información Técnica

- **Opción Correcta: B**
- **Justificación de la Opción Correcta:**
  Al inspeccionar la base de datos se confirma que la columna `password` almacena valores como `carlos123` o `admin123` en texto plano, violando el principio de almacenamiento criptográfico con

funciones unidireccionales lentas y salteadas (como BCrypt, Argon2id o PBKDF2). Además, `server.error.include-stacktrace=always` entrega al cliente la estructura de paquetes Java, líneas de código y excepciones JDBC detalladas cuando ocurre un fallo de base de datos. Se soluciona configurando `server.error.include-stacktrace=never` y codificando las credenciales con `BCryptPasswordEncoder`.

- **Análisis de Opciones Incorrectas:**
- **A es incorrecta:** No existen fallas de puertos ni agotamiento de memoria heap en la JVM.
- **C es incorrecta:** Argumento incoherente; no existen algoritmos cuánticos en Spring Boot 3 ni relación con HTTP 1.1.
- **D es incorrecta:** La consola H2 opera por defecto sin autenticación biométrica y la API se comunica fluidamente en JSON.